OnTimeAI: TP Integrador de Calidad de Software

Trabajo Práctico Integrador · Calidad de Software · UCC · 2026

Grupo OnTimeAI

26 de octubre de 2026

Resumen

OnTimeAI predice demoras de llegada de los vuelos que salen de Atlanta con un modelo LightGBM alimentado por un harvester de FR24, y las muestra en un dashboard web y en apps móviles. Son tres repositorios desplegados a Cloud Run. Este informe aplica a esa tesis el proceso de calidad de la materia: se declara el **Nivel 2 (prototipo parcial)**, se mide una línea de base el 22/10, se instalan en los tres pipelines gates de complejidad, bugs de análisis estático, duplicación y cobertura con trinquete, se incorporan k6 (stress y esfuerzo sostenido), un smoke post-deploy contra Cloud Run y una prueba de integración real contra Google Cloud Storage, se diseñan casos con clases de equivalencia, valores límite y tablas de decisión, se registra un defecto real detectado por el gate con contraste rojo/verde, y se mide una mejora de complejidad antes y después. Con todo aplicado, la cobertura del código desplegado pasó de 52,6 a 61,1 % en el Backend, de 50,7 a 64,8 % en el Scrapper y de 17,6 a 32,9 % en el Frontend, con 1.390 tests (antes 857); tres funciones salieron de la baseline de complejidad, y las pruebas nuevas encontraron cinco defectos más. Todos los números salen de los runs del pipeline y de `scripts/medir.py` del repositorio hub `OnTimeAI-Calidad`.

Repositorio del TP: https://github.com/santiago6124/OnTimeAI-Calidad. Repos del sistema: `OnTimeAI-Backend`, `OnTimeAI-Scrapper`, `OnTimeAI-Frontend` (mismo dueño).

1. Nivel declarado y estado inicial

Se declara **Nivel 2, prototipo parcial**. El sistema corre en producción y en las tiendas, pero la calibración del modelo, la fase 4 (inferencia desde el buffer FR24) y las apps siguen en validación, y la cobertura medida al inicio está lejos del 80 % que exige el Nivel 3.

Línea de base medida el 2026-10-22 sobre `origin/main` de cada repo, con `scripts/medir.py` (cobertura sobre **el código desplegado**: lo que copian los Dockerfiles; en el Frontend, `src/`):

Repo	Commit	Tests	Cobertura líneas	Cobertura ramas	Funciones con CC > 10	Promedio CC	CI	Sonar / smoke / stress
Backend	`416eef8`	544	52,6 %	42,8 %	21	5,93	sí (99 verdes / 1 rojo)	no / no / no
Scrapper	`2697284`	80	50,7 %	33,7 %	4	6,03	sí (36 / 1)	no / no / no
Frontend	`b68a6c2`	233	17,6 %	13,7 %	36	2,34	sí (100 / 0)	no / no / no

Lo que ya existía y se reutilizó: CI en cada push y PR, deploy automático a Cloud Run por Workload Identity, un vigía con alertas por Telegram, fallback FR24 → OpenSky, compare-and-swap por generación

sobre la base compartida, y la práctica de un test de regresión por cada fix desde septiembre (77 % de los fixes del Backend, 78 % del Scrapper).

2. Metas del nivel: cumplidas y no cumplidas

Aspecto (Nivel 2)	Exigido	Resultado	Cumplido
Pipeline	compilación, pruebas, análisis estático, cobertura, quality gate, reporte automático	los tres `ci.yml` corren ruff/ESLint, gate de complejidad, jscpd, tests con cobertura de líneas y ramas, diff-cover y un resumen en el job con artifacts	**sí**
Quality gate	bloquea bugs críticos, duplicación > 5 %, cobertura sobre código nuevo < 80 %	ruff E9/F63/F7/F82 y ESLint en error; jscpd `--threshold 5`; diff-cover `--fail-under 80`	**sí**, y el primer run del Backend lo demostró con un bug real
Cobertura sobre código nuevo	80 %	96 % en el PR del quality gate del Backend, 96 % en el del Scrapper, 94 % en el refactor	**sí**
Cobertura sobre código existente	60 %	Backend 52,6 → **61,1 %** de líneas; Scrapper 50,7 → **64,8 %**; Frontend 17,6 → 32,9 % (medido con todos los PRs aplicados, §6)	**Backend y Scrapper sí; Frontend no**; trinquetes instalados en los tres para que no baje
Smoke test	ejecutado por el pipeline tras el despliegue	`smoke.yml` con `workflow_run` tras `Deploy`: siete checks contra Cloud Run; verificado contra producción en 7 s	**sí**
Integración externa	una probada contra entorno real	compare-and-swap contra el GCS real en el bucket de QA `ontimeai-prod-qa`, por Workload Identity, en 5,6 s	**sí**
Stress y esfuerzo sostenido	ejecutado, con resultados y umbrales	k6 en `stress.yml`: smoke (1 VU), stress (0 → 20 → 50 VU, 3 min) y soak (20 VU, 10 min) contra la API con base fixture de 200 vuelos. PoC local: p95 135 ms / 11,1 s / 6,8 s; en CI (run 40011382323, soak de 10 min): p95 190 ms / 3,2 s / 1,7 s, 22 req/s, deriva del soak +10 %; 0 % de errores y checks 100 % en todos; umbrales `http_req_failed < 1 %`, checks > 99 %, p95 < 30 s (stress) y < 35 s (soak)	**sí**, y midió por primera vez la capacidad real (DEF-15)
Plan de pruebas y riesgos	versionado, criterios de entrada y salida, riesgos P × I, trazabilidad requisito → caso → defecto	`docs/PLAN_DE_PRUEBAS.md` 1.0, `RIESGOS.md`, `TRAZABILIDAD.md`	**sí**
Defectos	pasos, esperado, obtenido, vinculados a su caso	formulario "Defecto" en los tres repos; DEF-01 como issue #90; nueve retroactivos trazados a PR y test	**sí**
Herramienta nueva	cinco pasos	k6: `docs/HERRAMIENTA_K6.md` con requisitos, mercado, PoC, matriz ponderada y despliegue incremental	**sí**

3. Un eje por tema

3.1 Introducción a la calidad

Objetivos medibles desde el usuario (lead time de la predicción, AUC y Brier, p95 de la API, paridad web/iOS), desde el equipo (CI < 10 min, 80 % sobre código nuevo, trinquetes, un test por fix) y desde la organización (alerta en ≤ 30 min, rollback en ≤ 1 h, costo contenido): `docs/OBJETIVOS_DE_CALIDAD.md`. El plan de contingencia cubre FR24, GCS y Cloud Run con detección, modo degradado y recuperación, y apoya cada uno en un incidente real del proyecto (base corrupta del 2026-06-08, proyecto GCP muerto por billing el 2026-08-19): `docs/CONTINGENCIA.md`.

3.2 Fundamentos del testing

Los tests siguen Arrange-Act-Assert; el formulario de defecto exige escribir el resultado esperado antes de corregir. El contraste rojo/verde se hizo con un defecto real encontrado por el gate (§4). Los siete principios aplicados con evidencia propia están en `docs/PRINCIPIOS.md`: por ejemplo, el agrupamiento de defectos (19 de 45 fixes desde agosto en `api.py`) y la falacia de la ausencia de errores (un sistema sin bugs que marcaba el 46,6 % de los vuelos como demorados cuando debía marcar el 22 %).

3.3 Modelos de calidad y niveles de prueba

La matriz ISO/IEC 25010 con evidencia para las ocho características está en `docs/ISO25010.md`. Top 3: **fiabilidad** (la única que ya produjo pérdida de servicio), **adecuación funcional** (la calibración rota convierte una buena discriminación en alertas falsas) y **mantenibilidad** (la función más compleja del sistema es la menos cubierta). El mapa de niveles, unitario → integración con dobles → integración real → sistema (k6) → aceptación (smoke), y el despliegue verificado en local, CI, Cloud Run y tiendas están en `docs/PLAN_DE_PRUEBAS.md` §3.

3.4 Métricas y análisis estático

`docs/ANALISIS_ESTATICO.md` interpreta tamaño, complejidad (distribución por rango y concentración), duplicación, cobertura de sentencia y de decisión y bugs de análisis estático por repo, y explica la decisión de no depender de SonarQube para bloquear merges. La métrica de proceso (lead time de PR, fixes con test, duración del CI) y la de proyecto (PRs por semana) se calculan con `scripts/metricas_proceso.py`. Las mejoras medidas antes/después son tres refactors en el Backend (complejidad 16 → 5, 11 → 3 y 23 → 4; baseline 21 → 18, §6).

3.5 Pruebas dinámicas

`docs/CASOS_DE_PRUEBA.md` documenta 44 casos nuevos: la tabla de decisión de autenticación y autorización (9 filas), las clases de equivalencia del alta de usuarios, los valores límite de las bandas de riesgo (umbral y su doble, con sus vecinos) y de la cola de outcomes del Scrapper (29/30/31 min, 35 h y 36 h 1 min), más los casos de los gates, del smoke, del stress y de la integración. Todos pasaron contra el código actual: la API cumplía la tabla. La cobertura de decisión es la de ramas de la tabla de §1 y §6.

3.6 Aseguramiento de la calidad

Plan versionado con criterios de entrada y salida (`docs/PLAN_DE_PRUEBAS.md`), riesgos priorizados por probabilidad × impacto con cuatro de nivel 15-16 que ordenan el plan (`docs/RIESGOS.md`), prácticas

clasificadas en QA y QC (`docs/QA_VS_QC.md`), plantillas de caso y de defecto (`.github/ISSUE_TEMPLATE/defecto.yml` y `PULL_REQUEST_TEMPLATE.md` en los tres repos) y trazabilidad (`docs/TRAZABILIDAD.md`).

3.7 Automatización del testing

Pirámide: unitarias (pytest, vitest), API e integración (`TestClient`, dobles de GCS y FR24, integración real GCS), sistema (k6), aceptación (smoke post-deploy). Cada defecto corregido deja un test de regresión; el pipeline tiene al menos una ejecución verde y varias rojas que detectaron problemas reales (`docs/EVIDENCIA_CI.md`).

3.8 Herramientas del ecosistema

Mapa de herramientas por categoría (`docs/MAPA_HERRAMIENTAS.md`), k6 incorporado con los cinco pasos (`docs/HERRAMIENTA_K6.md`), registro de defectos en GitHub Issues con formulario, y stress y soak ejecutados por el pipeline.

4. Un defecto real, de rojo a verde

El primer run del PR del quality gate del Backend (run 40005299387) falló en el paso "Bugs críticos" con `F821 Undefined name download_faa_registry` en `feature_engineering_v7/adsb_lineage.py:367`: el setup del módulo llamaba a una función que nunca existió. Causa raíz: el módulo nació con la descarga llamada `download_aircraft_db`, la CLI conservó el nombre de un borrador, nadie volvió a correr `--setup` y ningún test pasaba por `main()`. Se registró como issue #90 con severidad, entorno, pasos, esperado, obtenido y evidencia (DEF-01). El test de regresión `tests/test_adsb_lineage.py` falla en el commit padre con `NameError` y pasa con el fix (`mediciones/evidencia/DEF-01-adsb-lineage-rojo-verde.txt`); el run siguiente (40006159701) quedó verde con 96 % de cobertura sobre las líneas cambiadas.

Otros rojos reales del TP: el gate de complejidad listó 21, 4 y 36 funciones en su primer run en cada repo; el smoke contra producción falló su séptimo check porque `/cascade` exige autenticación y la documentación del workspace lo daba por público (el criterio estaba mal, no la API).

5. Pipeline y evidencia

Los tres `ci.yml` corren, en este orden: gate de complejidad (ruff C901 con baseline-trinquete, radon como reporte), bugs críticos (ruff E9/F63/F7/F82 o ESLint + `tsc`), duplicación (jscpd ≤ 5 %), tests con cobertura de líneas y ramas con `fail_under` trinquete, cobertura sobre líneas cambiadas (diff-cover ≥ 80 %), resumen en el job y artifacts. Aparte: `smoke.yml` (tras `Deploy`), `stress.yml` (k6) e `integration.yml` (GCS real). La tabla completa de runs rojos y verdes con lo que detectó cada uno está en `docs/EVIDENCIA_CI.md`.

6. Evolución de las métricas

Medición "después" del 2026-10-23 con **todos los PRs del TP aplicados** en una rama local de integración por repo (`scripts/medir.py --ref medicion/despues`, misma definición y mismo script que la línea de base; se repite tras los merges):

Repo	Tests	Cobertura líneas	Cobertura ramas	Funciones con CC > 10	Funciones con CC > 20 (radon)	Promedio CC
Backend	544 → **771**	52,6 → **61,1 %**	42,8 → **52,0 %**	21 → **18**	11 → 10	5,93 → 5,71
Scrapper	80 → **130**	50,7 → **64,8 %**	33,7 → **46,5 %**	4 → 4	4 → 4	6,03 → 6,03
Frontend	233 → **489**	17,6 → **32,9 %**	13,7 → **24,9 %**	36 → 36	4 → 4	2,34 → 2,05

De dónde salió cada ganancia: en el Backend, 137 tests de caracterización sobre `faa_nas.py` (0 → 100 %), `predict.py` (42 → 99 %), `watchdog.py` (58 → 99 %) y `live_job.py` (65 → 99 %), 22 sobre el cliente AeroAPI, la paginación, el holding ADS-B y las features v8 de `live.py`, 27 de la tabla de decisión de autenticación y los valores límite de riesgo, y los tests del gate y del defecto DEF-01. En el Scrapper, 25 tests sobre `backfill_outcomes.py` (0 → 92 %), los clientes ADS-B (22 → 91-98 %) y las ramas de error del harvester, más 8 de valores límite. En el Frontend, 246 tests sobre `src/lib` (`api.ts` 36 → 100 %), los handlers de `src/app/api`, hooks y providers. Los 44 casos de equivalencia, límite y decisión ejercitan lógica que ya tenía tests, así que suman regresión y cobertura de decisión más que total. Lo que queda sin cubrir es conocido: `live_pull.py` (6 %) en el Backend, `harvester.py` (31 %) en el Scrapper, y páginas y componentes sin tests de render en el Frontend.

Mejora medida antes y después, con las herramientas del gate (PR Backend #94; detalle en `mediciones/evidencia/MEJORA-0{1,2,3}-*.md`):

Función (Backend)	ruff C901 antes → después	radon antes → después	Tests del módulo
`_coerce_parquet_frame`	16 → 5	C (18) → A (5)	51 verdes
`_upload_create_only`	11 → 3	C (19) → A (3)	51 verdes
`apply_lineage_fallback`	23 → 4	D (26) → A (4)	10 verdes
Baseline del gate	**21 → 18 exentas**		suite completa: 558 verdes

Se eligieron por combinar complejidad alta con cobertura previa alta (81 % y 93 %): la red de tests existía. En el tercer refactor el primer intento dejó afuera el final de la función y los tests lo marcaron en el acto. Por lo mismo no se tocó `live_pull.py:main` (72, 6 % de cobertura).

7. Análisis crítico

- **La cobertura del 60 % se alcanzó en el Backend (61,1 %) y en el Scrapper (64,8 %), no en el Frontend (32,9 %).** En el Frontend el hueco son las páginas y los componentes, que necesitan un entorno de navegador que quedó fuera por el orden de corte; lo que sí subió al 100 % es la capa de datos (`src/lib/api.ts`) y los handlers. Lo que impide retroceder quedó en los tres: trinquetes por repo (Backend 50 % combinado, Scrapper 46, Frontend 32/24/20/32, a subir tras los merges) y 80 % sobre cada línea nueva. El hueco mayor que queda es el mismo que concentra la complejidad: `live_pull.py:main` (72, 6 % de cobertura), que por eso no se refactorizó.
- **La definición de cobertura importa.** El primer run midió sólo los módulos que los tests importan (61 %), la primera medición local contó dos docenas de scripts de análisis que no se despliegan (39 %). El número que vale es sobre lo que copian los Dockerfiles (52,6 %), y está fijado en el `source` de coverage y en `scripts/medir.py` para que local y CI coincidan.

- **El quality gate funciona sin SaaS.** Se evaluó SonarQube Cloud; el gate que pide la cátedra quedó dentro del pipeline, y Sonar puede sumarse después como tablero consumiendo los mismos reportes.
- **Lo que detectaron los gates fue real y barato:** un `NameError` latente desde el primer commit de un módulo, en segundos de análisis estático.
- **Métricas de proceso:** lead time de PR por debajo de una hora porque una sola persona abre y mergea; el TP introduce la revisión cruzada como criterio de salida. En el Frontend sólo el 31 % de los fixes trae test porque son cambios de CSS y layout: ahí la pirámide necesita un nivel de navegador que quedó fuera del alcance por el orden de corte.
- **El stress midió por primera vez la capacidad de la API, y es baja:** un proceso sirve ~6,5 req/s del mix del dashboard en la Mac y 22 req/s en el runner, y satura con pocos usuarios concurrentes; p95 de 11 s (Mac) y 3,2 s (runner) en la rampa a 50 VUs, sin errores (hace cola, no se cae); el soak de 10 minutos no mostró fuga. Producción corre ese mismo proceso en una instancia. Los umbrales de k6 se fijaron al doble de lo medido para proteger que siga sin fallar, no como objetivo de servicio; el objetivo OU-03 queda como no cumplido y el hallazgo registrado (DEF-15) con su siguiente paso: cachear la ventana activa por ciclo y volver a medir con el mismo script.
- **Las pruebas nuevas encontraron cinco defectos** más allá del que detectó el gate: dos en el Backend (DEF-11, DEF-12), dos en el Frontend (DEF-13, DEF-14) y el de capacidad (DEF-15), todos en código que llevaba meses sin ejecutarse por un test. Quedan registrados con el formulario, con el caso que los reproduce, para corregirlos con contraste rojo/verde.
- **Riesgos que siguen abiertos:** FR24 como fuente única con canary pendiente (RP-01), la calibración (RP-03, ya con medición continua) y el plazo del TP (RY-01), que obligó a dejar afuera E2E en navegador y el stress contra Cloud Run.

8. Conclusiones

El proceso quedó instalado donde se ejecuta: cada PR de los tres repos pasa por un quality gate reproducible, cada deploy del Backend es verificado por un smoke contra Cloud Run, la integración crítica se prueba contra el servicio real y la API tiene por primera vez números de carga con umbrales. Las decisiones se apoyaron en mediciones propias (línea de base, defectos agrupados, complejidad concentrada, incidentes previos). En dos días la cobertura del código desplegado subió 8,5 puntos en el Backend, 14,1 en el Scrapper y 15,3 en el Frontend, y ese trabajo encontró cinco defectos reales además del que detectó el gate. Lo que no se cumplió (60 % en el Frontend, latencia de la API bajo carga) quedó medido, registrado y con el mecanismo para no retroceder. El nivel declarado se sostiene: prototipo parcial, con el pipeline de un prototipo avanzado.

Anexo: dónde está cada cosa

Entregable	Ubicación
Plan de pruebas, matriz ISO 25010, trazabilidad	`docs/PLAN_DE_PRUEBAS.md`, `docs/ISO25010.md`, `docs/TRAZABILIDAD.md`
Código de las pruebas	`tests/` de cada repo; `k6/` del Backend; `scripts/smoke.sh`; `tests/test_integracion_gcs.py` del Scrapper
Pipelines	`.github/workflows/{ci,smoke,stress,integration}.yml` en cada repo
Reportes automáticos	resumen del job y artifacts `calidad`, `complejidad`, `k6-resultados` de cada run
Integración externa	`docs/CONTINGENCIA.md`, PR Scrapper #22, `docs/SMOKE.md` del Backend
Registro de defectos	issues con label `defecto` en cada repo; índice en `docs/DEFECTOS.md`
Herramienta incorporada y matriz	`docs/HERRAMIENTA_K6.md`
Mediciones antes y después	`mediciones/2026-10-22-antes`, `mediciones/2026-10-23-despues`, `mediciones/evidencia/`, `mediciones/proceso-*.md`
Demostración	`docs/DEMO.md`